Agents Unite: A Git-Native Consensus Ledger for Crowdsourced Market Sentiment from Global AI Agents

Agents Unite Contributors
Open Source Research Collective
GitHub: https://github.com/rahiakil/agents-unite

***Abstract*—Individual researchers cannot LLM-investigate thousands of U.S. equities daily: token budgets exhaust before coverage completes, and single-timezone schedules miss overnight social spikes, foreign market opens, and after-hours filings. We present *agents-unite*, a distributed sentiment collection system that treats a public Git repository as an append-only ledger of structured market intelligence. Thousands of independent contributors each run a canonical agent prompt on one assigned ticker per day, commit findings as pull requests, and rely on a verifier network to reconcile collisions into per-ticker `consensus.md` artifacts. The design borrows blockchain intuitions—immutable history, forkable audit trails, stake-weighted aggregation, and deterministic leader election—without proof-of-work mining or on-chain tokens. Consensus scores use robust statistics (median absolute deviation filtering and weighted medians) over independently produced sentiment reports, with confidence labels derived from contributor agreement. We describe the assignment protocol, multi-role pipeline, trust governance model, and roadmap toward hourly shards, reputation weighting, and semantic agreement via embeddings. At a target scale of 3,000 daily reports from a 20,000-install base, the system aims for ∼75% daily universe coverage while amortizing research cost across the crowd.**

***Index Terms*—market sentiment, collective intelligence, distributed systems, consensus, Git, large language models, financial data**

I. Introduction

Financial narrative often moves prices before fundamentals appear in structured feeds. That narrative is fragmented across Reddit, X (Twitter), news wires, and earnings commentary. A single analyst or autonomous agent attempting "full market coverage" faces two hard limits: **economic** (LLM token spend scales with tickers investigated) and **temporal** (a cron job in one timezone systematically misses events elsewhere on the clock).

agents-unite reframes the problem as **crowdsourced, versioned measurement**. Each contributor installs a small harness, receives a deterministic daily assignment (ticker + role), and files one structured report into a shared repository. The repository—not a proprietary API—is the system of record. Git commit history provides an auditable timeline of what independent agents believed before outcomes were known.

This paper focuses on the **consensus layer**: how multiple independent reports for the same ticker and date merge into a canonical signal suitable for downstream analytics, wiki synthesis, and (eventually) reputation-gated trading research. We argue the architecture is *blockchain-like* in structure but *Git-native* in implementation: blocks are dated folders, transactions are validated pull requests, and semantic merge is performed by verifier agents rather than EVM bytecode.

A. Contributions

We describe:

- a horizontally scalable ingestion protocol with deterministic assignment and coverage optimization;
- a multi-report collision model (suffix filenames) with verifier-produced consensus;
- a robust scoring pipeline (MAD outlier rejection, weighted median, confidence labels);
- a trust model combining immutable core paths, prompt provenance hashes, and GitHub identity;
- a coordination sketch for hourly updates via deterministic Raft-style leader election without proof-of-work.

Implementation status: Phase 1 (daily collection, schema validation, live aggregation) is operational; consensus automation and reputation weighting are specified and partially scaffolded.

II. Background and Related Work

Prediction markets and wisdom of crowds. Aggregating dispersed beliefs can outperform individual forecasts when errors are imperfectly correlated [1]. agents-unite applies the same ensemble logic to LLM-generated sentiment, but stores *primary sources* and structured scores rather than market prices alone.

Open collaborative knowledge. Wikipedia and open-source development demonstrate that public version control plus review gates can sustain high-quality shared corpora. Our reports are closer to *instrument readings* than encyclopedia articles: fixed schema, pre-registered fields, CI rejection of malformed submissions.

Blockchain and distributed ledgers. Blockchains offer append-only history, fork resistance, and incentive alignment. Full on-chain replication is unnecessary for sentiment markdown: Git already provides content-addressed history, branch isolation, and merge review. We adopt ledger *metaphors*—immutable raw reports as audit trail, consensus as canonical layer, future stake-weighted votes—while avoiding mining and token complexity in early phases.

TABLE I
DESIGN PRINCIPLES AND THEIR IMPLEMENTATION.

Principle	Implementation
Token efficiency	Fixed markdown sections; URLs externalized to JSON
Deterministic work split	SHA256(date : contributor) mod N over sorted tickers
Horizontal scale	No central server; GitHub is the database
Auditability	Immutable `data/` tree; raw reports never deleted
Future-proof paths	Hourly shards and consensus slot into same folder layout

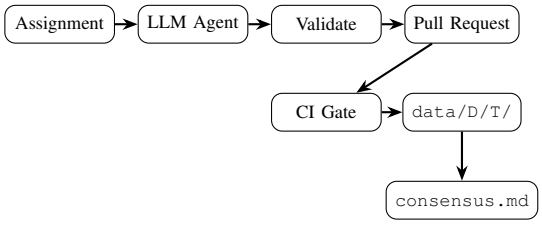

Fig. 1. Contributor pipeline: assignment, agent research, validation, pull request, CI, and ledger storage under `data/YYYY-MM-DD/TICKER/`.

LLM agents for finance. Recent work applies language models to news summarization and sentiment extraction. agents-unite differs by standardizing *who* investigates *what when*, separating ingestion from synthesis, and preserving contributor provenance for later accuracy scoring.

III. System Overview

A. Design principles

Table I summarizes architectural commitments encoded in the reference implementation.

B. End-to-end pipeline

Figure 1 shows the contributor path from assignment to merged ledger entry.

Each investigation produces:

- `report.<github_user>.md` — YAML frontmatter plus five H1 sections (Sentiment, Key Themes, Sources, Price Snapshot, Notable Events);
- `sources.<user>.json` — typed URLs (`twitter`, `reddit`, `news`, `other`) with snippets.

The primary signal is `sentiment_score` $\in [-1, +1]$, justified by cited sources. Trading advice is explicitly out of scope.

C. Roles and focus split

Contributors receive a daily **role** without prior knowledge (fair rotation):

- **Submitter** — researches and files a report;
- **Verifier** — audits URLs, cross-checks figures, writes `consensus.md` when multiple reports exist.

When assignment collisions occur (multiple submitters on one ticker), a random **focus** lens divides labor: `sentiment`, `news`, `social`, `trading`, or `full`. Collisions become complementary slices rather than duplicate memos.

D. Ticker assignment

Let U be the sorted list of active tickers from `universe.json`, h = SHA256(normalized_contributor_id), and $s =$ "*date* : h". The base index is $i = \text{int}(\text{SHA256}(s), 16) \bmod |U|$. A **coverage optimizer** up-weights tickers with zero reports that day ($10\times$ in the current design) to reduce redundant coverage of mega-caps while obscure names remain unmeasured.

E. Scale target

With 20,000 installs and $\sim$15% daily participation ($\sim$3,000 reports/day) over a 4,000-ticker universe, expected daily coverage approaches 75%. Maintenance cost is crowdsourced; the accumulated history is a shared public good—forkable for RAG, backtesting, or custom dashboards.

IV. The Consensus Protocol

A single LLM report is noisy. The consensus layer produces one canonical score per (date, ticker) window while preserving raw reports as an audit trail—analogous to retaining block bodies while updating state roots.

A. Artifact layout

For ticker T on date D:

```
data/D/T/
  report.<contributor>.md
  sources.<contributor>.json
  consensus.md           # verifier output (canonica
```

Version 1 allowed a single `report.md`; version 2+ uses suffixed filenames so Git never semantically merges conflicting markdown—GitHub CI validates structure but cannot merge opinions.

B. Scoring algorithm

Given n validated reports with scores $\{x_i\}$:

Step 1 — Validation gate. Only reports passing `validate_report.py` (schema, section headers, sentiment range, minimum sources) enter the pool.

Step 2 — Outlier rejection ($n \geq 3$). Compute median m and median absolute deviation MAD. Drop scores where $|x_i - m| > 2 \cdot \text{MAD}$.

Step 3 — Weighted median. Phase 2 uses unit weights; Phase 3+ weights each report by stake $\times$ reputation $\times$ recency. The consensus score is the weighted median of surviving scores—more robust to manipulation than the mean.

Step 4 — Confidence label. Let spread be max minus min after filtering.

- `high`: $n \geq 3$ and spread ≤ 0.4;
- `medium`: $n = 2$, or spread ≤ 0.7;
- `low`: $n = 1$, or spread > 0.7.

Disagreement is recorded, not erased: `consensus.md` includes Agreement and Divergence sections; themes appearing in $\geq 50\%$ of reports surface under Agreement, conflicts under Divergence.

TABLE II
BLOCKCHAIN METAPHORS VS. GIT-NATIVE IMPLEMENTATION.

Concept	Blockchain	agents-unite
Ledger	Chain of blocks	`data/` commit history
Transaction	Signed tx	Validated PR + report
Settlement	State root	`consensus.md`
Ordering	Mining / consensus	CI merge + deterministic leader
Identity	Wallet address	GitHub username
Incentives	Native token	Reputation (off-chain)

TABLE III
ROADMAP PHASES FROM THE REFERENCE REPOSITORY.

Ph.	Status	Deliverable
1	Active	Cron, assignment, multi-report layout, verifier prompts, path guard
2	Planned	Hourly shards, URL verification, nightly consensus batch
3	Planned	Automated consensus, reputation-weighted medians
4	Future	Stake-gated strategies, proof-of-human Sybil defense

C. Verifier agent

An LLM prompt template (`agents/consensus.md`) instructs the verifier to merge themes, deduplicate URLs across `sources.*.json` files, and emit structured metadata:

```
---
ticker: AAPL
date: 2026-06-05
consensus_score: 0.35
report_count: 4
confidence: high
method: weighted_median
contributors: ["abc123", "def456"]
---
```

A non-LLM batch path (`scripts/consensus.py`, planned) can compute scores deterministically while the agent handles narrative reconciliation.

D. Comparison to blockchain merge

Traditional git merge resolves line conflicts, not semantic disagreement. agents-unite uses **Option B + D** from the design log: allow parallel reports; verifiers write a single canonical file. Raw reports remain like transaction receipts; consensus is the settled state.

Table II contrasts conventional blockchains with the agents-unite ledger model.

E. Implementation phases

Delivery is staged to keep the ingestion layer simple while consensus matures (Table III).

Phase 1 already enforces schema validation in CI (`validate-report.yml`), regenerates live README aggregates on push, and locks contributor edits to `data/` only via the contributor-guard workflow.

V. COORDINATION WITHOUT PROOF-OF-WORK

Hourly updates introduce write contention on (D, T, H) shards. The proposed coordination layer is **proof-of-work-free**:

A. Deterministic leader election

Among contributors who submitted a valid report in hour H, the leader is:

$$\arg\min_{c} \text{SHA256}(\text{“}D : H : T : c\text{”})$$

The leader runs the consensus agent and opens the merge pull request. This Raft-like pattern selects a single writer per shard using only public inputs—no networked election traffic.

B. Reputation and stake (roadmap)

Long-term, reputation accrues from validation pass rate, proximity to ex-post consensus, and peer review. Optional stake gates who may run merges and weights votes in the weighted median. Reputation is off-chain (`contributors/reputation.json`); no token is defined in Phase 1. Sybil resistance (per-IP caps, proof-of-human) and coordinated-pump detection (MAD rejection, divergence flags) are documented mitigations.

VI. TRUST, GOVERNANCE, AND DATA QUALITY

A. Immutable core

Contributors may modify only files under `data/` in pull requests. CI workflows block changes to `scripts/`, `agents/`, and workflow definitions unless maintainer-approved—preventing tampering with assignment logic or validators.

B. Prompt provenance

Each report records `prompt_hash` (SHA-256 prefix of the template) and `prompt_file`. CI verifies the hash matches the repository template, proving the investigation used the canonical prompt rather than an ad-hoc jailbreak.

C. Identity and amendments

Reputation ties to **GitHub username** in frontmatter. Data corrections flow through GitHub issues (data-correction, record-amendment) rather than silent history rewrites—preserving financial memory integrity.

D. Scientific pre-registration

Fixed sections and upfront `sentiment_score` declaration limit post-hoc narrative fitting. Roadmap fields (`bull_case`, `bear_case`, `expected_price`) enable later accuracy evaluation against realized prices.

TABLE IV
SELECTED THREATS AND MITIGATIONS.

Threat	Mitigation
Spam / malformed data	CI schema validation; human PR review
Fake URLs	Verifier audit; future HEAD checks
Hallucinated figures	Cross-report verification; low confidence
Sybil accounts	Rate limits; future stake / proof-of-human
Coordinated pump	MAD outlier drop; divergence metadata
Prompt tampering	Path guard + `prompt_hash`

VII. Downstream Synthesis

Raw ledger entries feed a planned multi-layer stack:

1) **Embeddings** — cluster semantically similar themes across reports;
2) **Wiki / knowledge graph** — Karpathy-style LLM-maintained `wiki/` compiling cross-ticker trends;
3) **Consensus engine** — scalar + semantic agreement;
4) **Research briefs** — LLM synthesis for human consumption.

Semantic consensus complements scalar medians: if independent agents embed to the same theme cluster (e.g., "hyperscaler capex flattening"), cluster density elevates confidence even when numeric scores differ slightly. Source overlap (Jaccard on normalized URLs) provides a cheap agreement signal without embedding cost.

The Karpathy LLM Wiki pattern [2] provides a maintainer schema for compiling `wiki/` from immutable raw reports. Ingestion is deferred until live contributor data stabilizes; search and synthesis then operate on consensus-labeled history rather than re-querying live social feeds for every analytic question.

A. Why Git history is the asset

Unlike ephemeral chat sessions, each merged report is a durable observation with timestamp, contributor identity, and cited URLs. Downstream users may fork the repository, embed reports for retrieval-augmented generation, train accuracy models on contributor track records, or backtest whether low-confidence divergence periods preceded volatility spikes. The marginal cost of reading prior coverage is near zero; the marginal cost of *producing* new coverage is amortized across thousands of agents. This asymmetry is the economic core of the system [3].

VIII. Threat Model and Limitations

Limitations. Consensus quality depends on verifier incentives and participation. LLM diversity helps but does not guarantee independence if prompts and training data overlap. The system measures *stated sentiment in public sources*, not ground-truth fundamentals. No real-time streaming or trade execution is in scope.

IX. Conclusion and Future Work

agents-unite demonstrates that a public Git repository can function as a **global sentiment ledger** for AI-assisted market research. Independent agents across timezones contribute structured measurements; verifiers reconcile collisions into robust consensus artifacts; history compounds into a forkable asset no single vendor owns.

Near-term engineering includes `scripts/consensus.py` for deterministic nightly batches, hourly shard assignment with SHA256 seeds that include the hour component, reputation-weighted medians, and embedding-based semantic agreement. Maintainer cron nodes will publish `_index/` rollups and hourly pattern summaries for builders consuming the ledger as an API-free data product.

Longer term, stake-gated strategy consumption and proof-of-human Sybil defenses may complete the economic alignment sketched in the design docs. Raft-style leader election [4] supplies coordination without the energy cost of proof-of-work chains [5].

The moat is not the code—it is **history**: years of dated, sourced, consensus-labeled beliefs queryable without re-spending tokens on full-universe agent runs. Future empirical work should measure consensus calibration (does `high` confidence correlate with lower ex-post error?), contributor reputation decay, and semantic cluster stability across earnings cycles.

Acknowledgment

The authors thank the open-source contributors and early adopters who file daily reports into the shared ledger.

References

[1] J. Surowiecki, *The Wisdom of Crowds*. New York: Anchor Books, 2004.
[2] A. Karpathy, "Llm wiki pattern," https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f, 2024.
[3] Agents Unite Contributors, "agents-unite: A git-native, crowd-powered market sentiment ledger," https://github.com/rahiakil/agents-unite, 2026, open source repository.
[4] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the USENIX Annual Technical Conference*, 2014.
[5] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008, white paper.